

REFACTORTACTICS

Simulatore deterministico, snapshot e TurnLog

Stato logico, resolver, seed, replay e verifica divergenze

Documento: PDR-05

Versione: 0.1 - Consolidato iniziale

Data: 3 agosto 2026

Baseline tecnica: Unreal Engine 5.8

Specifica del nucleo autorevole indipendente dalla presentazione.

Controllo del documento

Campo	Valore
Stato	Bozza consolidata per revisione tecnica
Versione	0.1 - Consolidato iniziale
Data	3 agosto 2026
Autore	RefactorTactics / documentazione consolidata con supporto AI
Regola di prevalenza	Decisioni esplicite del progetto > requisiti consolidati > proposte PDR > ricerca web di supporto

Sommario esecutivo

Il resolver usa uno snapshot immutabile e ordinamenti stabili per produrre un nuovo stato e un TurnLog canonico. La presentazione riproduce eventi, ma non decide collisioni, danni, reazioni o obiettivi.

Decisioni consolidate

Stesso snapshot, RulesVersion, ContentManifestHash, ResolverConfigHash e seed devono produrre gli stessi hash finali.

Assunzioni operative

I formati di serializzazione e hash esatti sono da scegliere durante Fondazioni; devono essere versionati prima dei replay persistenti.

Indice del documento

1. Obiettivi determinismo
2. Stato logico
3. Snapshot schema
4. Ordine stabile
5. Seed e random
6. Resolver
7. Event model
8. Replay
9. Hash e divergenze
10. Test

1. Obiettivi determinismo

Il determinismo richiesto è applicativo: stessa build/rules version, stesso snapshot e stesso seed devono produrre lo stesso stato finale e lo stesso log canonico. Non si richiede che ogni effetto visivo sia identico né che la fisica real-time decida risultati competitivi.

2. Stato logico

Dominio	Contenuto minimo	Vincoli
MapState	Celle, archi, hazard, occupazione, revisioni	Nessun riferimento a componenti visuali.
Units	ID, cella, facing, HP, risorse, status	Ordinabili per StableUnitId.
Effects	ID, source, target, durata, stack	Ordine e trigger espliciti.
Objectives	stato, posizione, punteggio	Aggiornati in fase definita.
Turn	numero, fase, rules version	Monotono e serializzabile.
RNG	seed turno e stream ID	Nessun FMath::Rand globale.
Log	eventi, reason, before/after essenziali	Replay ed explainability.

3. Snapshot schema

```
FRTTurnSnapshot
RulesVersion
ContentManifestHash
MatchId / TurnNumber
TurnSeed
MapSnapshot
  GraphRevision, Cells[], Edges[], Hazards[]
Units[]          // sorted StableUnitId
Effects[]        // sorted Phase, Priority, EffectId
Objectives[]
AcceptedIntents[] // sorted Team, Player, Unit
ResolverConfigHash
```

Lo snapshot è un value object logico. Può essere costruito da UObject/Actor, ma il resolver preferisce strutture pure e compatte per ridurre dipendenze e facilitare test headless.

4. Ordine stabile

- Non dipendere dall'ordine di TMap/TSet per risultati competitivi.
- Ordinare esplicitamente per fase, priorità, iniziativa se prevista e StableId come tie-break finale.
- Definire regole per conflitti di movimento, target multipli e reazioni simultanee.
- Registrare nel log la chiave di ordinamento o il reason code quando influisce sull'esito.

5. Seed e random

Ogni turno riceve un seed autorevole. Le estrazioni casuali devono usare stream nominati o derivati in modo stabile, ad esempio Hash(TurnSeed, ActionId, RollKind). In questo modo aggiungere un VFX casuale non sposta le estrazioni di hit/crit.

Regola

Le abilità competitive non accedono direttamente a random globale. Ogni roll significativo produce un evento con stream key, range e risultato.

6. Resolver

Il resolver consuma snapshot e intenti, produce eventi e un nuovo stato. L'implementazione iniziale può essere sincrona sul server; la separazione in fasi consente successiva parallelizzazione solo dove l'ordine osservabile resta invariato.

```

Validate snapshot invariants
-> Build ordered action queue
-> For each phase
    -> Resolve triggers/reactions
    -> Resolve micro-steps or impacts
    -> Append canonical events
    -> Apply events to working state
-> Finalize objectives/cooldowns/KO
-> Compute StateHash + LogHash
-> Publish result

```

7. Event model

Evento	Dati essenziali
MoveStarted	UnitId, from, intended destination
MoveStep	UnitId, from, to, step index, cost
MoveBlocked	UnitId, attempted cell, reason, blocker
AbilityDeclared	ActionId, source, AbilityId, target policy
DamageApplied	source, target, amount, damage type, modifiers
StatusChanged	target, StatusId, stacks/duration, reason
EnvironmentChanged	cell/edge, before, after, source
UnitKO	UnitId, source/reason
ObjectiveUpdated	ObjectiveId, before, after

8. Replay

Il replay di turno ricostruisce la presentazione dal TurnLog e dallo stato iniziale. Per audit competitivo si conserva anche lo snapshot o un riferimento verificabile. Le animazioni sono consumatori del log: se un montage termina tardi, il risultato non cambia.

9. Hash e divergenze

- ContentManifestHash identifica definizioni usate dal match.
- ResolverConfigHash identifica ordine fasi e parametri classificati.
- StateHash normalizza e serializza campi logici in ordine stabile.
- LogHash copre gli eventi canonici, non timestamp real-time.
- In caso di mismatch il client scarta simulazioni locali e applica il risultato server; telemetry registra versione e hash.

10. Test

- 1 Golden test: snapshot fixture -> log e state hash attesi.
- 2 Permutation test: inserire unità/effects in ordine diverso e ottenere output uguale.
- 3 Repeat test: 1.000 esecuzioni con stesso seed, zero differenze.
- 4 Seed test: cambiare seed modifica solo eventi random correlati.
- 5 Frame-rate test: playback a 30/60/144 FPS non modifica stato finale.
- 6 Packaged server test: confrontare hash tra build Development e Shipping compatibile.